United 431 Request Header Size Guidance

Overview

This document summarizes our current guidance for investigating and mitigating the HTTP 431 "Request Header Fields Too Large" issue observed in United's browser-based headless Ping SDK / DaVinci flow when using the custom domain. United reported that requests to the custom domain were still carrying the full cookie set, increasing header size and resulting in 431 responses.

Current understanding

On the browser-based headless DaVinci SDK path, there is not a direct equivalent to the widget's `originCookies` whitelist behavior. In this model, the browser's native cookie behavior and the surrounding application architecture are the main levers that determine what cookies and headers are sent on requests.

Because of that, this issue should be approached primarily as a browser / cookie-scope / request-routing problem rather than as an SDK setting problem.

Likely cause

The most likely issue is that too many cookies are being attached to the custom-domain requests, either directly by the browser because of cookie scope and credentialed requests, or by an intermediary such as Akamai or another proxy that is forwarding unnecessary headers upstream.

We have seen cases where the practical constraint was request header size at the intermediary layer rather than DaVinci itself requiring those cookies.

Important clarification

CORS still needs to be correct for credentialed browser requests, but CORS by itself does not resolve a 431 if the underlying issue is oversized request headers.

Recommended remediation options

1. Route DaVinci traffic through a backend-for-frontend or proxy

This is typically the cleanest option. In that pattern, the browser no longer sends its full cookie jar directly to DaVinci, and the server can forward only the cookies or headers that are actually required upstream.

2. Tighten cookie scope on the application side

Review cookie domain, path, and retention settings so unrelated application cookies are not attached to DaVinci or custom-domain requests. Avoid broad cookie scoping on the same host used for authentication traffic.

3. Limit credentialed requests to only the calls that require session reuse

When `credentials: include` is used, the browser will send cookies that match the target domain. Restricting credentialed requests to the calls that truly need an existing session can reduce header size.

4. Review intermediary behavior

If Akamai or another proxy is in the path, strip nonessential cookies before forwarding the request to Ping / DaVinci. This is an important check because the effective failure point may be at the intermediary layer rather than inside DaVinci itself.

5. Use a custom domain aligned to the same eTLD+1 as the application where possible

This is the recommended pattern to reduce cross-site cookie issues and minimize the need for additional browser workarounds.

Recommended next diagnostic steps

We recommend capturing a HAR from a failing browser session and identifying exactly which cookies are being attached on the custom-domain requests that return 431. From there, determine whether those cookies are being added because of normal browser credential behavior, broad cookie scoping, or an intermediary in the path.

Suggested working sequence

- Capture a HAR on a failing request path.

- Identify the specific request returning 431 and enumerate the attached cookies / headers.
- Determine whether the added cookies are coming from browser cookie scope, `credentials: include`, or an intermediary such as Akamai.
- Apply the best-fit remediation: BFF / proxy first, cookie-scope cleanup second, and aligned custom domain third.

Conclusion

Our current recommendation is to treat this as an architectural and request-shaping issue rather than as a simple SDK configuration gap. The most robust path is to inspect the exact failing requests, reduce unnecessary cookies on the auth path, and, if needed, introduce a BFF or proxy so only the required headers and cookies are forwarded upstream.
